

Docker: основы технологии

Что такое Docker

Docker — платформа для создания, доставки и запуска приложений в изолированной среде, которая называется **контейнер**.

Главная идея: приложение работает в контейнере со своими настройками и зависимостями, но не создаёт нагрузку на систему больше, чем обычный процесс — в отличие от полноценных виртуальных машин (VirtualBox и т. п.), которые эмулируют всё «железо» и требуют намного больше ресурсов.

Контейнер

Контейнер — способ упаковать приложение вместе со всем, что ему нужно для работы (код, библиотеки, настройки), чтобы оно одинаково работало в любой среде: на ноутбуке разработчика, на сервере, в облаке.

Аналогия: если нужно перевезти вещи, вы кладёте их в коробку — коробка (контейнер) защищает содержимое и позволяет переносить всё как единое целое.

Ключевое отличие контейнеров от виртуальных машин: контейнеры используют общее ядро операционной системы хоста, а не создают собственное. Поэтому они гораздо легче и быстрее виртуальных машин.

Из чего состоит Docker

Компонент	Что делает
Docker daemon (сервер)	Фоновый процесс на хосте, который управляет образами, контейнерами, сетями, томами
Docker CLI (клиент)	Командная строка для общения с daemon (<code>docker run</code> , <code>docker build</code> и т. д.)
Docker Engine	Общее название «движка» Docker — daemon + CLI вместе
Docker-контейнер	Запущенный изолированный процесс со своей файловой системой, портами и т. д.
Docker-образ (image)	Шаблон только для чтения, из которого создаётся контейнер. Можно хранить в системе контроля версий, передавать другим
Docker registry (реестр)	Хранилище образов. Пример — Docker Hub (аналог GitHub, но для образов)
Dockerfile	Текстовый файл с инструкциями для сборки образа
Docker Compose	Инструмент для запуска приложений, состоящих из нескольких контейнеров сразу
Docker Swarm	Инструмент для объединения нескольких Docker-хостов в один кластер

Клиент и даемон общаются через **REST API** (программный интерфейс для передачи запросов). Они могут быть как на одной машине, так и на разных (клиент подключается к удалённому серверу).

Зачем нужен Docker

- **Одинаковая среда** везде — решает проблему «у меня на компьютере работает, а на сервере нет».
- **Портативность** — контейнер легко переносится между системами.
- **Изоляция** — контейнеры не мешают друг другу.
- **Экономия ресурсов** — контейнеры легче виртуальных машин.
- **Быстрое масштабирование** — контейнеры быстро создаются и удаляются.
- Удобен для **микросервисной архитектуры** (каждый сервис — свой контейнер) и интеграции в DevOps-процессы.

Установка

Есть Community Edition (бесплатная, для обучения) и Enterprise Edition. Устанавливается через **Docker Desktop** — есть версии под Windows/macOS/Linux.

Проверить, что Docker установлен и работает:

```
docker version
```

Команда выводит версию клиента (Client) и сервера (Server) отдельно.

Основные команды Docker

Команда	Что делает
<code>docker pull image:tag</code>	Скачивает образ из реестра. Если <code>tag</code> не указан — берётся <code>latest</code>
<code>docker run [опции] image:tag</code>	Создаёт и запускает контейнер из образа
<code>docker exec -it <container> /bin/sh</code>	Выполняет команду внутри уже запущенного контейнера (например, открывает терминал)
<code>docker cp <container>:/path /local/path</code>	Копирует файл из контейнера на локальную машину (и наоборот — поменяв местами)
<code>docker ps</code>	Показывает список запущенных контейнеров (<code>-a</code> — показать все, включая остановленные)
<code>docker build [опции] PATH</code>	Собирает новый образ из <code>Dockerfile</code>
<code>docker stop <name hash></code>	Останавливает контейнер
<code>docker start <name hash></code>	Запускает остановленный контейнер заново
<code>docker rm <name hash></code>	Удаляет контейнер
<code>docker images</code>	Показывает список загруженных образов

Опции команды docker run

- `-i` — интерактивный режим (открывает `STDIN` , поток ввода);
- `-t` — создаёт псевдотерминал (`tty`);
- `-d` — запускает контейнер в фоне (режим демона), без привязки к терминалу.

Для `docker exec` не обязательно писать весь хеш/имя контейнера — достаточно первых 4–5 символов, если этого хватает, чтобы Docker однозначно понял, о каком контейнере речь.

Порты и тома (volumes)

По умолчанию контейнер изолирован — снаружи к нему не подключиться. Чтобы дать доступ к сервису внутри контейнера (например, к базе данных), нужно **пробросить порт**:

```
docker run -p 27017:27017 -d mongo
```

Слева — порт на вашем компьютере, справа — порт внутри контейнера. Число слева можно менять, справа — обычно берут «стандартный» порт сервиса (у MongoDB это `27017`).

Чтобы данные не терялись при удалении контейнера, используют **volumes (тома)** — сохранение данных не внутри контейнера, а на диске хоста:

```
docker run -p 27017:27017 -v e/dbstorage:/data/db -d mongo
```

Так даже если контейнер удалить и создать новый с тем же volume — данные останутся.

Пример практической работы

```
docker run -it hello-world
```

Запускает тестовый контейнер, который проверяет, что Docker работает. Если образа `hello-world` ещё нет локально — Docker сам сначала скачает его (`pull`), потом запустит (`run`).

Докеризация приложения

Разберём на практике весь путь: обычное Flask-приложение (выводит «Hello World!» в браузере) → упаковываем его в Docker-образ → запускаем как контейнер.

1. Подготовка проекта

Создаём папку проекта и переходим в неё:

```
mkdir flask-docker-app
cd flask-docker-app
```

В проекте нужно 2 файла: `app.py` — сам скрипт приложения, и `requirements.txt` — список пакетов (библиотек), которые использует проект.

Создаём виртуальное окружение и активируем его:

```
python -m venv env
source env/bin/activate
```

Устанавливаем Flask:

```
pip install Flask
```

Flask вместе с собой ставит ещё несколько зависимых пакетов. Чтобы зафиксировать их все в `requirements.txt`:

```
python -m pip freeze > requirements.txt
```

Файл `requirements.txt` — это список пакетов с точными версиями. Он нужен, чтобы при сборке образа Docker поставил внутри контейнера ровно те же версии библиотек, что и у вас локально.

Код `app.py` — простейший веб-сервер на Flask:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello World!"

if __name__ == '__main__':
    app.run()
```

При обращении на главный маршрут (`@app.route('/')`) приложение отдаёт браузеру текст «Hello World!». Запускаем локально:

```
python app.py
```

Приложение станет доступно по адресу `http://localhost:5000/`. Останавливаем сервер сочетанием `Ctrl+C` — дальше переходим к докеризации.

2. Файл Dockerfile

Создаём в корне проекта файл с именем `Dockerfile` (без расширения). Комментарии в нём пишутся через `#` в начале строки.

```
# Docker-команда FROM указывает базовый образ контейнера
# Наш базовый образ – Linux с предустановленным python-3.10
FROM python:3.10

# Задаём переменную окружения
ENV APP_HOME /app

# Задаём рабочую директорию внутри контейнера
WORKDIR $APP_HOME

# Копируем остальные файлы в рабочую директорию контейнера
COPY . .

# Устанавливаем зависимости внутри контейнера
RUN pip install -r requirements.txt

# Открываем порт, на котором работает приложение внутри контейнера
EXPOSE 5000

# Запускаем приложение внутри контейнера
ENTRYPOINT ["python", "app.py"]
```

Основные инструкции Dockerfile

Инструкция	Что делает
FROM	Базовый образ, поверх которого всё строится. Формат <code>образ:тег</code> . Всегда первая строка в Dockerfile
ENV	Задаёт переменную окружения, доступную и в следующих инструкциях, и внутри самого контейнера
WORKDIR	Рабочая директория для всех следующих команд (<code>RUN</code> , <code>CMD</code> , <code>COPY</code> и т. д.) . Можно указывать несколько раз
COPY	Копирует файлы из контекста сборки (папки проекта) внутрь образа. Нельзя скопировать файл, который лежит вне этой папки
RUN	Выполняет команду прямо во время сборки образа и сохраняет результат
EXPOSE	Сообщает Docker, какой порт слушает процесс внутри контейнера (используется вместе с <code>-p</code> в <code>docker run</code>)
ENTRYPOINT	Команда, которая запускается при старте контейнера — «точка входа» в приложение
CMD	Похоже на ENTRYPOINT, но легко переопределяется аргументами при запуске <code>docker run</code> . Если инструкций CMD несколько — сработает только последняя
ADD	То же самое что COPY, но умеет ещё брать файлы по URL и автоматически распаковывать архивы. На практике обычно проще и безопаснее пользоваться COPY
VOLUME	Объявляет каталог как том (для сохранения данных вне контейнера). На практике чаще используют ключ <code>-v</code> в команде <code>docker run</code>
MAINTAINER	Указывает автора образа (посмотреть можно через <code>docker inspect</code>). Устаревшая инструкция, встречается редко

3. Правка app.py перед сборкой

Чтобы приложение внутри контейнера было доступно снаружи, нужно указать `host='0.0.0.0'` — это значит «слушать все сетевые интерфейсы», а не только внутренний адрес контейнера:

```
if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0')
```

4. Сборка и запуск образа

Собираем образ (выполнять из корня проекта):

```
docker build --no-cache . -t ваш_логин/flask-docker
```

Здесь `ваш_логин` — имя пользователя на hub.docker.com, а `flask-docker` — произвольное имя образа.

При сборке Docker показывает шаги в консоли. Если запустить сборку повторно без изменений в файлах — неизменившиеся шаги будут помечены как `CACHED` и выполняться мгновенно: Docker переиспользует уже собранные слои образа вместо того, чтобы делать всё заново.

Запускаем контейнер из собранного образа, пробросив порт `3000` на хосте к порту `5000` внутри контейнера:

```
docker run -itd -p 3000:5000 ваш_логин/flask-docker
```

Приложение станет доступно на `http://localhost:3000/` — сервис снаружи достаём с порта 3000, хотя внутри контейнера он всё ещё работает на 5000.

Play with Docker — бесплатная песочница в браузере (docker.com/play-with-docker) для экспериментов с Docker без установки чего-либо на свой компьютер.

Docker Compose

Docker Compose — инструмент для запуска приложений, состоящих из нескольких контейнеров одновременно. Вместо того чтобы вручную прописывать `docker run` для каждого контейнера, все сервисы описываются в одном YAML-файле `docker-compose.yml`, а управляются одной командой.

Пример: проект использует сразу две базы данных — PostgreSQL и Redis, каждая в своём контейнере. Вот как это описывается в `docker-compose.yml`:

```
version: '3'
services:
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
  postgres:
    image: postgres:12
    environment:
      POSTGRES_DB: rest_app
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 567234
    ports:
      - "5432:5432"
    volumes:
      - ./postgres-data:/var/lib/postgresql/data
```

Здесь заданы два сервиса:

- **redis** — образ `redis:alpine`, порт контейнера `6379` проброшен на такой же порт хоста;
- **postgres** — образ `postgres:12`, порт `5432`, заданы переменные окружения (имя базы, пользователь, пароль) и volume — данные базы сохраняются в локальной папке `./postgres-data`, а не только внутри контейнера.

С такой конфигурацией оба сервиса запускаются одной командой: `docker-compose up -d`.

Основные команды Docker Compose

Команда	Что делает
<code>docker-compose up</code>	Создаёт и запускает контейнеры для всех сервисов из файла, а также сеть для их взаимодействия
<code>docker-compose down</code>	Останавливает и удаляет контейнеры, сети и тома, созданные командой <code>up</code>
<code>docker-compose ps</code>	Список контейнеров, созданных <code>up</code> , и их статус
<code>docker-compose logs</code>	Логи контейнеров, созданных <code>up</code>
<code>docker-compose exec <сервис> <команда></code>	Выполнить команду в запущенном контейнере (например, <code>docker-compose exec redis redis-cli</code>)
<code>docker-compose build</code>	Пересобрать образы, если менялись файлы сервисов

Добавление своего приложения в Compose

Чтобы Flask-приложение из примера выше тоже управлялось через Compose вместе с Redis и PostgreSQL, добавляем в тот же файл ещё один сервис — `app`, который собирается из Dockerfile, лежащего рядом с `docker-compose.yml`:

```
version: '3'
services:
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
  postgres:
    image: postgres:12
    environment:
      POSTGRES_DB: rest_app
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 567234
    ports:
      - "5432:5432"
    volumes:
      - ./postgres-data:/var/lib/postgresql/data
  app:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - redis
      - postgres
```

Новый сервис `app`: собирается из Dockerfile в текущей папке (`build: .`), слушает порт `5000`, и через `depends_on` указано, что он должен стартовать после Redis и PostgreSQL.

Теперь одна команда `docker-compose up` поднимает сразу три контейнера — Redis, PostgreSQL и само приложение, — и все они видят друг друга в общей сети, которую создаёт Compose.

Docker Compose особенно полезен для локальной разработки: он позволяет одной командой воссоздать на своей машине окружение, близкое к продакшн, без ручного запуска каждого контейнера по отдельности.